

65 - Identity tutorial from scratch

ASP.NET Core Identity Tutorial

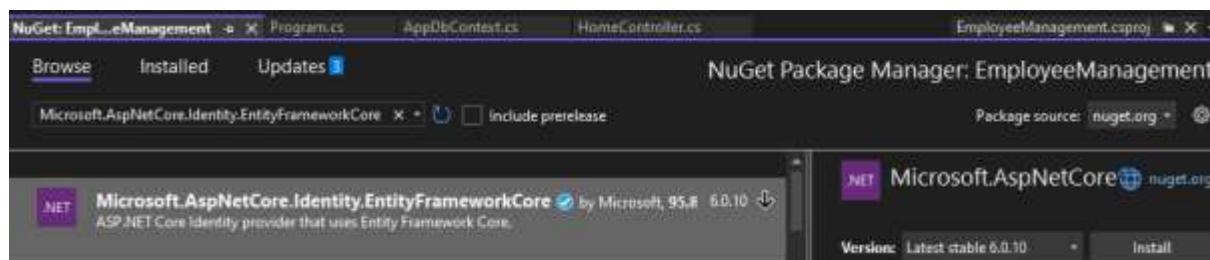
ASP.NET Core Identity is a membership system

- Create, Read, Update and Delete user accounts
- Account confirmation
- Authentication & Authorization
- Password Recovery
- Two-factor authentication with SMS
- Supports external login providers like Microsoft, Facebook, Google etc
- and much more

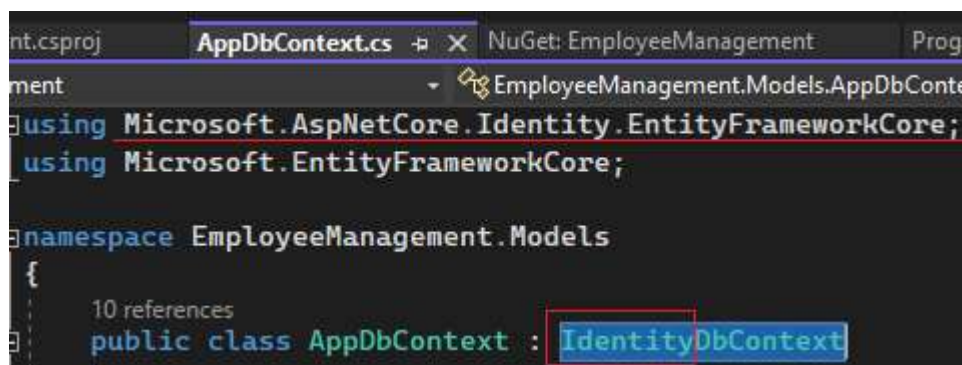
За целта първо унаследяването трябва да е IdentityDbContext, вместо само DbContext

Налага се да инсталирам, за да извикам необходимия namespace

Microsoft.AspNetCore.Identity.EntityFrameworkCore:



Identity притежава комплект от всички свойства за управление на вградената таблица БД, в случая SQL server.



Когато направим няколко последователни проследявания с Go to Definition ->

IdentityDbContext, ще видим, че всъщност той е разширение и се унаследява от по-ограничения DbContext, като го включва в себе си, затова не е необходимо да дефинираме допълнително още веднъж DbContext. IdentityDbContext включва DbContext + повече Identity функционалност.

Step 1 : Your application DbContext class must inherit from IdentityDbContext

```
public class AppDbContext : IdentityDbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options)
        : base(options)
    {
    }

    public DbSet<Employee> Employees { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.Seed();
    }
}
```

Следваща стъпка е да добавим неговата Identity services в Startup.cs -> Configure Services

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddDbContextPool<AppDbContext>(
        options => options.UseSqlServer(_config.GetConnectionString("EmployeeDBConnection")));

    services.AddIdentity<User, Role>();

    services.AddMvcOption<IdentityUser, IdentityRole>()
```

Ще ги наречем IdentityUser, IdentityRole, които изискват namespace:

```
using Microsoft.AspNetCore.Identity;

services.AddIdentity<IdentityUser, IdentityRole>()

services.AddMvcOption<IdentityUser, IdentityRole>()
```

Go to Definition:

```

...public IdentityUser(string userName);

...public virtual DateTimeOffset? LockoutEnd { get; set; }
...public virtual bool TwoFactorEnabled { get; set; }
...public virtual bool PhoneNumberConfirmed { get; set; }
...public virtual string PhoneNumber { get; set; }
...public virtual string ConcurrencyStamp { get; set; }
...public virtual string SecurityStamp { get; set; }
...public virtual string PasswordHash { get; set; }
...public virtual bool EmailConfirmed { get; set; }
...public virtual string NormalizedEmail { get; set; }
...public virtual string Email { get; set; }
...public virtual string NormalizedUserName { get; set; }
...public virtual string UserName { get; set; }
...public virtual TKey Id { get; set; }
...public virtual bool LockoutEnabled { get; set; }
...public virtual int AccessFailedCount { get; set; }

...public override string ToString();

```

Присъстват важни компоненти като телефон, имейл, парола, но няма примерно за град или пол. За да ги добавим, добавяме клас, в който да се дефинират те и после този клас да се унаследи в IdentityUser class.

IdentityRole е аналогичен.

За да може информацията от тези два системни класа IdentityUser, IdentityRole да бъде обработвана, добавяме AddEntityFrameworkStores, в който вкарваме нашия клас.

```

system class, system class
services.AddIdentity<IdentityUser, IdentityRole>()
    .AddEntityFrameworkStores<AppDbContext>();
custom class as generic parameter

```

```

services.AddIdentity<IdentityUser, IdentityRole>()
    .AddEntityFrameworkStores<AppDbContext>();

```

Step 2 : Add ASP.NET Core Identity Services

```

public void ConfigureServices(IServiceCollection services)
{
    services.AddDbContextPool<AppDbContext>(options =>
        options.UseSqlServer(_config.GetConnectionString("EmployeeDBConnection")));

    services.AddIdentity<IdentityUser, IdentityRole>()
        .AddEntityFrameworkStores<AppDbContext>();

    services.AddMvc().AddXmlSerializerFormatters();
    services.AddScoped<IEmployeeRepository, SQLEmployeeRepository>();
}

```


Step 3 : Add Authentication middleware

```
public void Configure(IApplicationBuilder app, IHostingEnvironment env)
{
    if (env.IsDevelopment())
    {
        app.UseDeveloperExceptionPage();
    }
    else
    {
        app.UseExceptionHandler("/Error");
        app.UseStatusCodePagesWithReExecute("/Error/{0}");
    }

    app.UseStaticFiles();
    app.UseAuthentication();

    app.UseMvc(routes =>
    {
        routes.MapRoute("default", "{controller=Home}/{action=Index}/{id?}");
    });
}
```

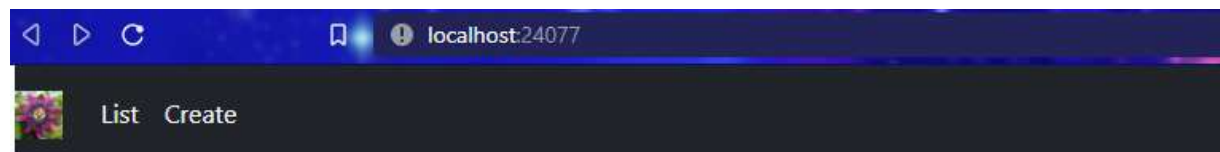
Добавяме Authentication middleware. Редът отново е много важен - трябва да е преди MVC:

```
app.UseStaticFiles();
app.UseAuthentication();
app.UseMvc(routes => //app
```

Компилираме, за да се уверим, че нямаме грешки:

```
Output
Show output from: Build
1>EmployeeManagement -> D:\Exercises\EmployeeManagement\bin\Debug\net6.0\EmployeeManagement.dll
===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped =====
```

Появява се страницата с грешка:



An error occurred while processing your request.
The support team is notified and we are working on the fix.
Please, contact us on support@abv.bg

```
<h3>
    An error occurred while processing your request.
    <br />
    The support team is notified and we are working on the fix.
</h3>
<h5>Please, contact us on support@abv.bg</h5>
<hr />
```

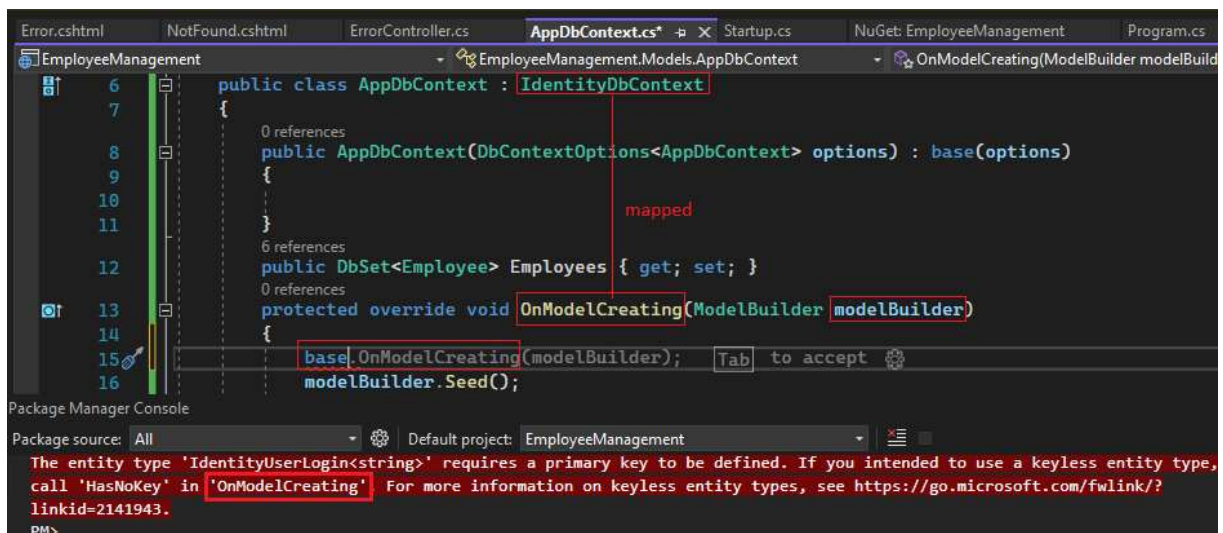
Step 4 : Generate ASP.NET Core Identity Tables

Add-Migration

View-> Other Windows -> Package Manager:

Add-Migration AddingIdentity

Keys of IdentityTables are mapped in OnModelCreatingMethod of this IdentityDbContext system class.



```
public class AppDbContext : IdentityDbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options)
    {
    }
    public DbSet<Employee> Employees { get; set; }
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.Seed();
    }
}
```

Package Manager Console

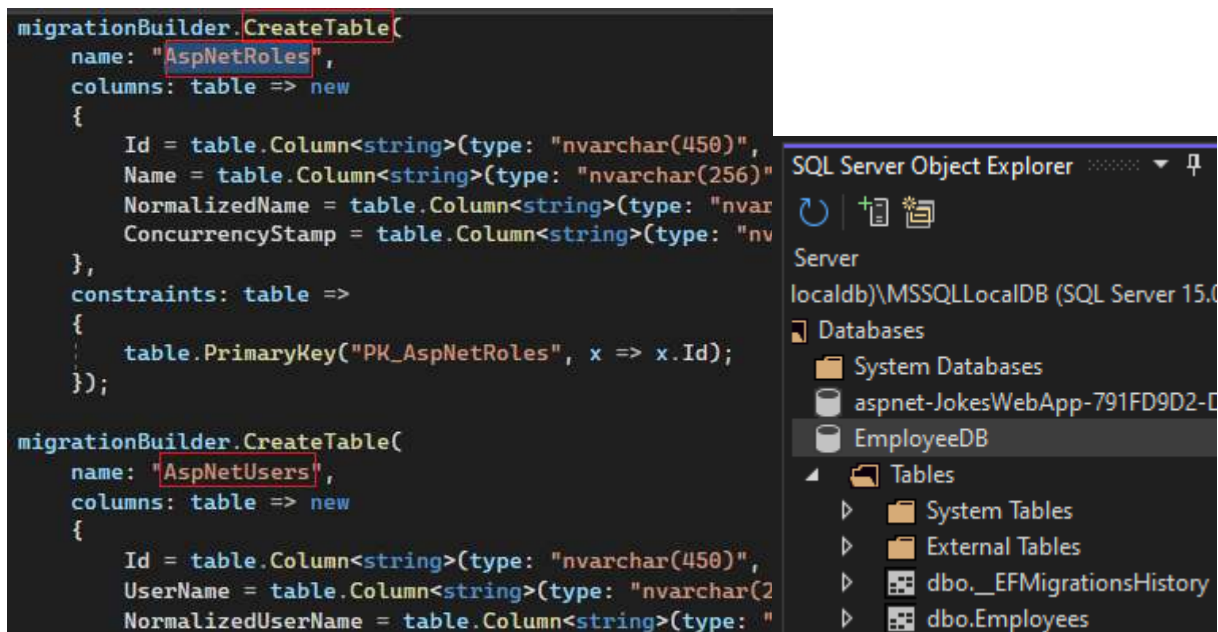
Package source: All Default project: EmployeeManagement

The entity type 'IdentityUserLogin<string>' requires a primary key to be defined. If you intended to use a keyless entity type, call 'HasNoKey' in 'OnModelCreating'. For more information on keyless entity types, see <https://go.microsoft.com/fwlink/?linkid=2141943>.

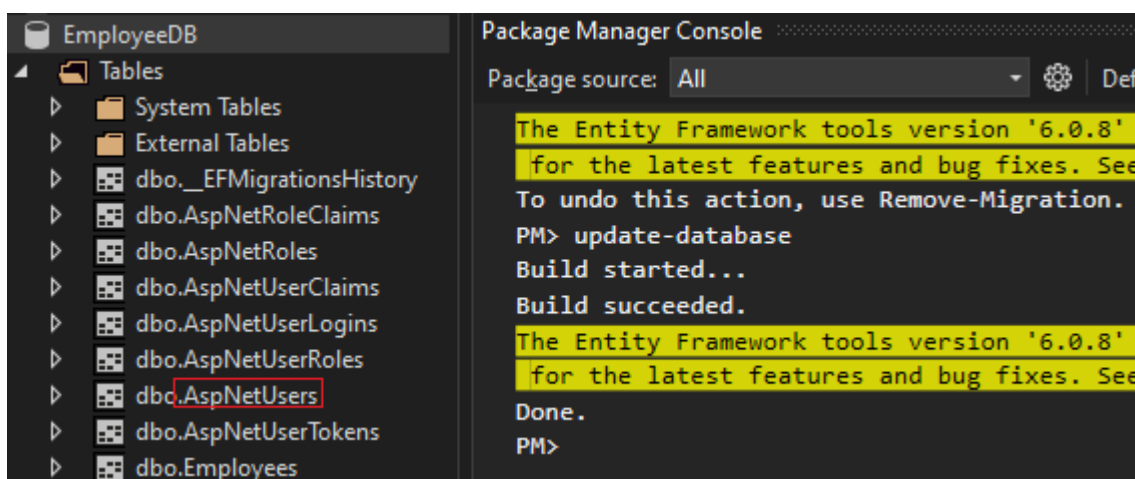
PM>

```
PM> add-migration AddingIdentity
Build started...
Build succeeded.
The Entity Framework tools version '6.0.8' is older than that of the runtime '6.0.10'.
bug fixes. See https://aka.ms/AAc1fbw for more information.
To undo this action, use Remove-Migration.
PM>
```

Миграцията създава няколко таблици:



Към момента имаме само 2 БД, но когато обновим миграцията update-database, веднага се зареждат няколко нови.



->View Data

id	UserName	NormalizedU...	Email	NormalizedEm...	EmailConfirmed	PasswordHash	SecurityStamp	ConcurrencySt...	PhoneNumber	PhoneNumber...	TwoFactorDna...	Locko...
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Нейните колони съответстват в системно заложените в клас IdentityUser, аналогично за IdentityRole. Към момента в тях няма допълнителна информация като град и пол, това предстои да се направи като разширение.

Step 4 : Generate ASP.NET Core Identity Tables

Add-Migration

Update-Database